Digital Signal Processing Exercise Sheet 2

Systems

Script: Chapter 2 (Systems)

Learning goals

- Use the operator view $y = S\{x\}$ and decide for a given system whether it is static or dynamic (memory), causal, linear and time-invariant.
- Understand how a two-point lambda controller with integrator works, why it produces a limit cycle, and predict its period from the loop dead time.
- *Bonus (optional):* design numerical test harnesses that *reveal* a property and know their pitfalls (impulse-only tests, small amplitudes, boundary effects); model engine subsystems as dynamic systems: inertia (first-order lag), wall film, transport delay, switching sensor.

Definitions used on this sheet

For a system $y = S\{x\}$ (continuous or discrete time):

- *memoryless (static)*: $y(t_0)$ depends only on $x(t_0)$;
- *causal*: $y(t_0)$ depends only on $x(t)$, $t \le t_0$; equivalently, if $x_1(t) = x_2(t)$ for $t < t_0$, then $y_1(t) = y_2(t)$ for $t < t_0$;
- *linear*: $S\{ax_1 + bx_2\} = aS\{x_1\} + bS\{x_2\}$ for all inputs and constants (superposition = additivity + homogeneity);
- *time-invariant*: $y(t) = S\{x(t)\} \Rightarrow y(t - t_0) = S\{x(t - t_0)\}$ for every shift t_0 (discrete time: integer shifts n_0).

Linearity and time invariance are previewed here; Chapter 3 shows why their combination (LTI) is so powerful.

Exercise 2.1 Properties by hand

PAPER

a) Decide for each system whether it is memoryless, causal, linear and time-invariant (table with yes/no):

S1: $y(t) = 2x(t)$	S2: $y(t) = x^2(t)$	S3: $y[n] = x[n+1] - x[n]$
S4: $y[n] = \frac{1}{3}(x[n] + x[n-1] + x[n-2])$	S5: $y(t) = x(t - T_d)$, $T_d > 0$	S6: $y(t) = \lvert x(t)\rvert$
S7: $y(t) = \int_{-\infty}^{t} x(\tau)\,\mathrm{d}\tau$	S8: $y[n] = n\,x[n]$	S9: $y[n] = x[n] + 0.5$
S10: $y[n] = x[2n]$	S11: $n_k = 1/T_k$ (Sheet 1)	S12: $U = 0.5 + 0.4\tanh\frac{1-\lambda}{0.005}$

b) Prove your answers formally for S6 (linearity), S8 (time invariance) and S10 (causality and time invariance). For S9, show that the system is not linear, but that the *difference* of two outputs depends linearly on the difference of the inputs (*incrementally linear*).

c) An engineer computes the mean speed of one crank revolution as the arithmetic mean of the 58 per-tooth speeds $1/T_k$, treating the gap period like a normal tooth period. Compute the relative error at constant speed. After correcting the gap ($3/T_{\text{gap}}$), is the arithmetic mean of the per-tooth speeds equal to $60/T_{\text{rev}}$? Which property of S11 is responsible, and what is the error for a speed fluctuation of $\pm 1\,\%$ (sinusoidal)?

Exercise 2.2 Closed lambda loop

WOKWI

Project `code/wokwi/sheet02_lambda/`: Arduino Uno and the custom chip `lambda-probe`. Pin 9 sends the fuel command as PWM (`analogWrite`, 490 Hz, duty $= q/2$), A0 reads the sensor voltage `O2`, A1 the ground truth `LAM` $= \lambda/2$. The chip's sliders set the unmetered air (slider -10 in %, default $+5$ %) and the transport delay (default 200 ms).

How a two-point controller works. The goal is the stoichiometric mixture $\lambda = 1$. The ECU does not know the fuel quantity q^* that achieves it, because part of the air is not measured (leaks, sensor and injector tolerances, ageing): with unmetered air a, $\lambda = (1+a)/q$ and $q^* = 1 + a$. The switching sensor (S12 of Exercise 2.1) delivers only *one bit* of information: $U > 0.45$ V means "rich" ($\lambda < 1$, too much fuel), $U < 0.45$ V means "lean" ($\lambda > 1$, too little fuel) – but not *how* rich or lean. The controller therefore uses the simplest possible rule. In every controller period $T_s = 10$ ms:

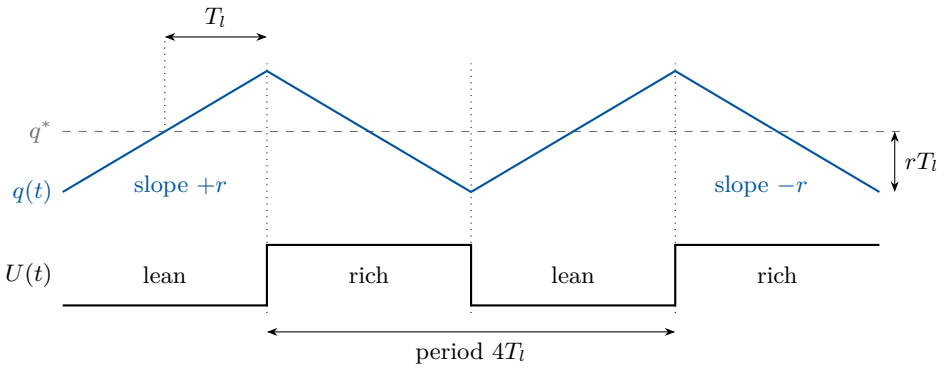
- sensor rich $\Rightarrow$ decrease q by rT_s (ramp down with slope $-r$),
- sensor lean $\Rightarrow$ increase q by rT_s (ramp up with slope $+r$),
- optionally, at the moment the sensor *switches*, an additional jump of q by P towards the other side ($-P$ on lean→rich, $+P$ on rich→lean).

"Two-point" means that the controller output (here the slope $\dot{q}$) takes only two values, $\pm r$, like a relay or a thermostat. "With integrator" means that q is the integral of this output: as long as the sensor reports the same state, q keeps moving in the same direction, so it always moves towards q^*, whatever the value of a. On average the controller therefore finds q^* by itself (integral action).

Why it oscillates. The sensor sees a change of q only after the transport delay T_d and the two lags (wall film $\tau_f = 20$ ms, sensor $\tau_s = 30$ ms), i.e. after an effective dead time $T_l \approx T_d + \tau_f + \tau_s$. When q crosses q^*, the sensor still reports the old state for T_l, and q keeps ramping in the wrong direction: it overshoots by rT_l. After the switching, q needs another T_l to return to q^*, and the sensor sees this crossing only T_l later. Each half period therefore lasts $2T_l$:

$$T_{\text{period}} \approx 4\,T_l, \qquad q = q^* \pm rT_l, \qquad \lambda \approx 1 \mp \frac{rT_l}{q^*}.$$

The oscillation neither grows nor decays – a *limit cycle*. The jump P removes part of the overshoot immediately after each switching, so q reaches q^* earlier (by P/r); the period shortens to $4T_l - 2P/r$.



Limit cycle without jump ($P = 0$): U is the sign of $q - q^*$, delayed by T_l.

a) Implement the two-point controller with integrator ($r = 0.2$/s) in `loop()`. Start the simulation and observe the limit cycle in the Serial Monitor (`u_o2`, `lambda`, `q`). Copy 5 s of data into a spreadsheet (or into JSLinux, see the bonus section) and plot λ and U; compare with Fig. 1.3 of the script and with the sketch above. Predict period and λ range for $T_d = 200$ ms and check them.

b) Measure the oscillation period (column `period_ms`) for transport delays of 100, 200, 400 and 800 ms and plot period over T_d. Determine slope and intercept and interpret them.

c) Move the air slider from +5 % to −10 % and to +10 %. Describe the transient and the new mean value of q.

d) Add the jump `PJUMP` = 0.03, then reduce `RATE` to 0.1 with `PJUMP` = 0.025. Compare period and λ amplitude. Why can no two-point controller make the oscillation disappear, and what would a continuous (PI) lambda controller need?

Deliverables

2.1: property table, proofs. 2.2: sketch, plot of λ and U over 5 s, period over T_d (table + plot), results of c) and d). Bonus 2.B1 (optional): test program, property table with numerical evidence, $h[n]$ of the LTI systems. Bonus 2.B2 (optional): program, values of τ, response times, prediction formula and comparison table.

Bonus: JSLinux (optional)

For interested students: the following exercises identify unknown systems by experiment and simulate the lambda path offline in C. Bonus Exercise 2.B2 c) derives the limit cycle of Exercise 2.2 in detail.

Setup

The following exercises run in JSLinux, a Linux system in the browser. If you have not done Bonus Exercise 0.B1, set it up now (about 10 min):

1. Open `https://bellard.org/jslinux`, start the *x86 Alpine Linux* (console) virtual machine and check the compiler with `gcc -version` (or `tcc -v`).
2. Upload the course headers `engine_signals.h`, `asciiplot.h`, `csvio.h` (`code/common/`) and the files of the exercise (e.g. `code/jslinux/sheet02/`) with the upload button below the terminal; check with `ls` that they arrived. Headers and program must be in the same directory.
3. Compile and run, e.g.

```
gcc -O2 -o blackbox blackbox.c -lm
./blackbox
```

4. The VM forgets all files when the page is reloaded: keep your sources on your own computer and upload them again. Text from Wokwi (Serial Monitor) gets into the VM with `cat > data.csv`, paste, `Ctrl-D`; `export_file data.csv` downloads a file from the VM to your computer.

Exercise 2.B1 Black-box system identification JSLINUX

The header `code/jslinux/sheet02/mystery.h` contains five unknown systems

$$\texttt{void Sk(const double *x, double *y, int N);} \quad k = 1, \ldots, 5$$

(input $x[0..N-1]$, $x[n] = 0$ outside, output $y[0..N-1]$). One of them is LTI; the others violate exactly one or two of the properties of Exercise 2.1. **Do not read the header** – find the properties by experiments, as you would with a supplier's ECU function delivered as object code. The program `code/jslinux/sheet02/blackbox.c` plots the responses to unit impulses at $n_0 = 30$ and $n_0 = 31$ and contains a (weak) homogeneity test with scaled impulses as an example.

a) Superposition test: compare $S\{ax_1+bx_2\}$ with $aS\{x_1\}+bS\{x_2\}$ for random signals and random weights, at amplitude 1 and 0.01. Why is the impulse test of the template not sufficient, and why two amplitudes?

b) Shift test: compare $S\{x[n-k]\}$ with $y[n-k]$ for $k = 1 \ldots 4$. Why are the first and last samples of the test signal set to zero?

c) Causality test: two inputs that agree for $n < n_0$ and differ afterwards.

d) Memory test: two inputs that differ only at n_0. Why must the test be repeated for several n_0?

e) Report a property table with the numerical evidence (largest deviation of each test). For the LTI systems determine $h[n]$ from the plots and give the difference equation. Why does an "impulse response" not characterise the other systems?

Exercise 2.B2 The engine as a dynamic system JSLINUX

Program: `code/jslinux/sheet02/engine_dyn.c`.

a) *Crankshaft inertia.* With fuel quantity q (1 = nominal), the engine speed obeys $\tau\,\dot{n} = Kq - n$, $\tau = 0.4\,\mathrm{s}$, $K = 2000\,\mathrm{rpm}$. Simulate a fuel step $1 \to 1.5$ at $t = 0.5\,\mathrm{s}$ with the Euler method ($T = 1\,\mathrm{ms}$) and with the exact discretisation $n[k] = a\,n[k-1] + (1-a)Kq[k-1]$, $a = \mathrm{e}^{-T/\tau}$ (derive it!). Determine τ from the 63 % point and compare both methods with the analytic solution at $t = 0.6\,\mathrm{s}$. Is the engine (fuel → speed) static or dynamic?

b) *Lambda path.* Implement `lp_step()` (one step = 1 ms) – it is the model inside the Wokwi chip `lambda-probe`: $q \to$ wall film (first-order lag, 20 ms) $\to \lambda = (1 + a_{\mathrm{air}})/q_f \to$ transport delay T_d $\to$ sensor lag (30 ms) $\to U = 0.5 + 0.4\tanh((1-\lambda)/0.005)$. Classify each block (static/dynamic, linear, TI). Simulate fuel steps $1.00 \to 1.10$ and $1.00 \to 1.20$ ($a_{\mathrm{air}} = +5\,\%$, $T_d = 200\,\mathrm{ms}$) and measure the time until U crosses 0.45 V. Which observations prove that the chain is nonlinear? Under which operating condition is it not time-invariant on a real engine?

c) *Closed loop – offline model of Exercise 2.2.* A two-point controller with integrator changes q with slope $-r$ while the sensor reports rich ($U > 0.45\,\mathrm{V}$) and $+r$ while lean (period 10 ms, 8-bit PWM). Derive the period and the λ amplitude of the resulting oscillation as functions of T_d, r and the lag time constants (hint: a ramp passing a first-order lag τ is delayed by τ). Implement the controller in `closed_loop()`, compare with your prediction for $T_d = 50 \ldots 800\,\mathrm{ms}$, and explain what happens to the mean of q when the unmetered air changes. Extend the prediction to a controller with an additional jump $\pm P$ at every switching of the sensor.